Práctica 2 – Análisis de datasets y aplicación de técnicas de IA en Python

El desarrollo de esta práctica se centra en el análisis de conjuntos de datos y en la aplicación de técnicas de inteligencia artificial en el contexto de un **Sistema de Detección de Intrusiones en Red (NIDS, Network Intrusion Detection System)**.

- **Detección de intrusiones:** El objetivo principal es identificar y alertar sobre posibles intrusiones, sin responder ni actuar directamente ante ellas. Al tratarse de un problema de detección basado en datos con características definidas, diversas técnicas de inteligencia artificial resultan especialmente adecuadas, incluyendo clasificación, detección de anomalías y clustering.
- **Red:** El enfoque está a nivel de red, en contraste con los sistemas basados en host (HIDS). Nos centramos únicamente en ataques que provienen del exterior y que afectan a la red en su conjunto o a equipos específicos, operando a nivel de red. Esto significa que no se abordan amenazas que actúan directamente sobre equipos individuales, como malware, virus o ransomware.

La práctica consiste en:

1. **Análisis del dataset** (entrega de código)
2. **Entrenamiento de modelos de IA** (entrega de una memoria justificativa)
3. **Presentación** oral del trabajo en horario de clase.

La práctica se realiza en **grupos de 4 personas.**

Durante el desarrollo de la práctica se empleará el **dataset CIC-IDS2017**, generado de forma sintética en un entorno controlado, lo que elimina el problema de la sensibilidad de la información a la hora de hacer un conjunto de datos público.

Están disponibles para descargar tanto las capturas de tráfico originales como los flujos de tráfico generados, lo que evita el problema del procesamiento de grandes cantidades de datos, de la presencia de contenido encriptado en el cuerpo de los paquetes, y de posible sobregeneralización de los modelos.

El conjunto de datos contiene 5 días de tráfico en horas laborales, durante las cuáles se han realizado los siguientes ataques:

Categoría	Ataque
Fuerza Bruta	Fuerza Bruta SSH
	Fuerza Bruta FTP
Denegación de Servicio - DoS	Slowloris
	Slowhttptest
	Hulk
	GoldenEye
Denegación de Servicio - DDoS	LOIT
Vulnerabilidades	Heartbleed - Puerto 444
Ataques Web	Fuerza Bruta (Ataque Web)
	XSS
	SqlInjection
Ataques de infiltración	Dropbox
	Cooldisk
Botnet	Botnet ARES
Escaneo de Puertos	Port Scan

1. Análisis del dataset

Descarga el dataset de (https://www.unb.ca/cic/datasets/ids-2017.html).

Para la versión en csv del dataset, está disponible tanto la versión normal (TrafficLabeling) como una versión ML-ready (MachineLearningCVE), que tiene ciertas características, que pueden influir en el entrenamiento de modelos de machine learning, eliminadas.

Importante **descargar la versión normal**, ya que muchas de estas características pueden ser útiles para el análisis, agregación, preprocesamiento. Un ejemplo de esto sería el timestamp de los flujos, que se desecha en la versión MachineLearningCVE.

Crea un jupyter notebook en el que vayas a realizar todo el preprocesado y análisis sobre este dataset, siguiendo las siguientes pautas:

- **Identifica características que puedan introducir sesgos**.
- **Identificar características categóricas**. Para estos casos, es necesario darles una codificación numérica, para ello se pueden emplear las estrategias de codificación más habituales (codificación simple, ordinal, One-Hot-Encoding…). Justificar la decisión adoptada.
- Genera una gráfica con la distribución de clases del dataset y **analiza el desbalanceo** de clases presente en el dataset tanto teniendo en cuenta clasificación binaria como multiclase.
- Obtener las **10 características más y menos importantes**. Por ejemplo, empleando PCA, t-SNE, UMAP…
- **Segmenta el conjunto de datos por clases**. Por ejemplo, empleando PCA, t_SNE, UMAP... Observa cómo la varianza de distintos tipos de ataques depende de distintas características.
- **Genera la Matriz de Correlación del dataset**. Deberías de ver cómo todas las características derivadas de la misma variable tienen alta correlación. También deberías observar cómo ciertas características dependen linealmente del tamaño en paquetes del flujo. Identifica las características correspondientes a estos casos.

1.1. Preprocesado

El preprocesado del conjunto de datos es un paso fundamental antes de entrenar cualquier técnica de Inteligencia Artificial. Para ello se recomienda realizar los siguientes pasos:

- **Agrupa valores de características**. Si se considera necesario o interesante agrupar por rangos los valores de una categoría. Justificar la decisión adoptada.
- **Gestionar missing values** y **duplicado.** Para estos casos, estudiar las diferentes alternativas (suprimir, imputación simple, imputación guiada, imputación informada…). Justificar la decisión adoptada.
- **Gestionar outliers.** Detectar si existen y tomar la decisión de qué hacer con ellos aplicando las estrategias más habituales: análisis visual, IQR, Percentiles, Z-score, IsolationForest… Justificar la decisión adoptada.
- **Escalar/Normalizar los datos.** Para asegurar que todas tengan el mismo peso o acotar el rango de valores. Las estrategias más habituales para aplicar son: estandarización, escalado mínimo-máximo, escalado robusto, escalado absoluto, escalado por la norma… Justificar la decisión adoptada.
- Se recomienda usar windowing **sólo si el modelo a entrenar es temporal**.
- Se recomienda usar **selección de características o reducción de dimensionalidad** (PCA, Random Forest, o cualquier otro). Justificar la decisión adoptada.
- Realizar el split train/validación/test. **Importante** seleccionar un ratio adecuado para asegurar una buena comparativa entre modelos. Además, si existiese un fuerte desbalanceo entre clases, las probabilidades de que un split subrepresente una clase son significativas. En este caso, asegurarse de usar un split **estratificado** por clases. Justificar la decisión adoptada.
- Adicionalmente, se puede mitigar la subrepresentación de ciertos ataques en el dataset usando técnicas de **undersamplig** y **oversampling**. Empleando por ejemplo: **SMOTE** o **ADASYN**. Justificar el método empleado. El uso de las técnicas adicionales aquí descritas se evaluará positivamente.

2. Entrenamiento de modelos de IA

El objetivo de este apartado consiste en el **entrenamiento de dos modelos de IA**, la **comparativa** entre ellos, y el **análisis de los resultados** obtenidos, además del análisis de cualquier otra técnica auxiliar usada.

Cada grupo deberá de seleccionar dos modelos, uno de cada lista, sobre los cuáles deberá desarrollar su trabajo. **Los modelos son exclusivos entre grupos**, es decir, cada grupo tendrá dos modelos únicos.

Lista 1	Lista 2
AdaBoost (s) – *sklearn*	**Hoeffding Adaptive Tree (s)(o) –** *river*
SVM (s) – *sklearn*	***Autoencoder (AE) (u)*** – *PyTorch*
OC-SVM (u) – *sklearn*	**CNN (s)** – *PyTorch*
HDBSCAN (u)(c) – *sklearn*	**LSTM (s)** – *PyTorch*
XGBoost (s) – *xgboost*	**Transformer NN (s)** – *PyTorch*
SOM maps (u)(c) – *minisom*	**GAN (u)** – *PyTorch*
ANN (s) – *PyTorch*	

Listas de modelos y librerías asociadas (u = unsupervised; s = supervised; c = clustering; o = online)

El entrenamiento de los modelos se hará siguiendo un entrenamiento estándar basado en las divisiones de *train*, empleado para el entrenamiento de los modelos, *validation*, empleado para búsquedas de hiperparámetros opcionales, y *test*, empleado principalmente para reportar métricas y analizar el rendimiento en la memoria.

Los modelos para entrenar deberán resolver un problema de clasificación binaria: Es decir, la salida final de cada modelo será una predicción donde cada flujo será etiquetado como positivo (presencia de ataque) o negativo (tráfico benigno).

Adicionalmente, y siempre que la capacidad y arquitectura del modelo lo permita, los modelos pueden resolver un **problema de clasificación multiclase**, dónde la salida final del modelo será una predicción sobre a que clase del dataset etiquetado original pertenece el flujo de entrada.

Las métricas tienen que ser apropiadas para cada uno de los casos y, en el caso del problema binario, deberán ser usadas en todos los modelos del grupo, siempre y cuándo resuelvan un problema binario, para una correcta y justa

comparación entre estos. Para el caso de clasificación binaria, las métricas recomendadas consisten en:

- Accuracy
- Recall
- Precision
- F1-Score
- AUC-ROC

Para el caso de clasificación multiclase, en entornos en los que existe un claro desbalanceo entre clases, las clases de más baja frecuencia dentro de las divisiones de validación y test apenas tendrán impacto sobre métricas tradicionales. Por eso, es importante usar métricas *Macro Average*, que evalúan por igual cada una de las clases sin tener en cuenta la cantidad de muestras correspondiente a esa clase. Las métricas recomendadas son:

- Macro Average Recall
- Macro Average Precision
- Macro Average F1-Score

La evaluación de este trabajo se centrará en la correcta implementación y análisis del entrenamiento de los modelos y técnicas, no en su rendimiento final, siempre y cuando la falta de rendimiento esté correctamente razonada a nivel de datos, técnicas, o el dominio de la tarea.

3. Entrega

Parte **1. Análisis del dataset**, entregar a través del campus virtual:

- Un **jupyter notebook conteniendo el código**, correctamente estructurado y comentado, en dónde se realice el análisis del dataset. Las celdas deben de contener la salida de esta ya ejecutada, permitiendo dentro del notebook ver los gráficos y prints pertinentes. (características más comunes, diagrama de clases, …). Se valorará la adecuación y claridad del código.

Fecha límite de entrega: 20/03/2026 a las 23:59.

Parte **2. Entrenamiento de modelos de IA**, entregar a través del campus virtual:

- Una **memoria** bien estructurada, escrita y justificada dónde se refleje tanto el procedimiento usado para el entrenamiento del modelo básico y el modelo complejo, cómo un análisis de los resultados obtenidos y una comparación entre los resultados generados por ambos modelos. Se valorará la calidad del contenido de la memoria.

Fecha límite de entrega: 17/04/2026 a las 23:59.

4. Presentación

El día **21/04/2026** en horario de clases cada grupo procederá con la **presentación de TODO el trabajo realizado**. Para ello, cada grupo dispondrá de entre 8 y 10 minutos donde tendrán que exponer y justificar las decisiones tomadas. Excederse de este límite podría llevar a una penalización en la evaluación del grupo.

Todos los miembros del grupo deben participar equitativamente en la exposición, encargándose de exponer una parte del trabajo, por lo que cada uno dispondrá de, aproximadamente, 2 minutos. De darse el caso de que un integrante del grupo no exponga, o su exposición sea irrelevante para el contenido del trabajo, su nota podría verse penalizada respecto a la nota del grupo.

Se recomienda **preparar una serie de diapositivas** para realizar la exposición. En caso de emplear la memoria de forma directa, debéis verificar que dispone de la calidad adecuada para ser proyectada y leída por el resto de los presentes.

El eje central de la presentación serán las técnicas y modelos empleados, junto con los desafíos encontrados a la hora de usarlos. También deberá

de cubrir el análisis de los datos, su preprocesado y los resultados obtenidos, realizando una comparativa entre modelos.